



PDF Download
2882903.2882944.pdf
15 March 2026
Total Citations: 42
Total Downloads: 1074

 Latest updates: <https://dl.acm.org/doi/10.1145/2882903.2882944>

RESEARCH-ARTICLE

Query Planning for Evaluating SPARQL Property Paths

NIKOLAY YAKOVETS, York University, Toronto, ON, Canada

PARKE GODFREY, York University, Toronto, ON, Canada

JAREK GRYZ, York University, Toronto, ON, Canada

Open Access Support provided by:

York University

Published: 26 June 2016

[Citation in BibTeX format](#)

SIGMOD/PODS'16: International
Conference on Management of Data
June 26 - July 1, 2016
California, San Francisco, USA

Conference Sponsors:
SIGMOD

Query Planning for Evaluating SPARQL Property Paths

Nikolay Yakovets Parke Godfrey Jarek Gryz
York University
Toronto, Canada
{hush,godfrey,jarek}@cse.yorku.ca

ABSTRACT

The extension of SPARQL in version 1.1 with *property paths* offers a type of *regular path query* for RDF graph databases. Such queries are difficult to optimize and evaluate efficiently, however. We have embarked on a project, WAVEGUIDE, to build a cost-based optimizer for SPARQL queries with property paths. WAVEGUIDE builds a query plan—which we call a *waveplan* (WP)—which *guides* the query evaluation. There are numerous choices in the construction of a plan, and a number of optimization methods, so the space of plans for a query can be quite large. Execution costs of plans for the same query can vary by orders of magnitude. A WP’s costs can be estimated, which opens the way to cost-based optimization. We demonstrate that the plan space of WAVEGUIDE properly subsumes existing techniques and that the new plans it adds are relevant.

1. INTRODUCTION

Graph data is becoming rapidly prevalent with the rise of the Semantic Web, social networks, and data-driven exploration in life sciences. There is need for natural, expressive ways to query over these graphs. Standards are coming into place for this. The Resource Description Framework (RDF) [21] provides a data model for graph data. An RDF *store* is a set of *triples* that describes a directed, edge-labeled multi-graph. A triple, $\langle s, r, o \rangle$, denotes an *edge* from node “s” (the *subject*) to node “o” (the *object*), with the edge labeled by “r” (the *role*, also called *label* or *predicate*). The SPARQL query language [20] correspondingly provides a formal means to query over RDF stores. A query defines sub-graph match criteria; its evaluation over an RDF store returns all embedded sub-graphs meeting the criteria. For example, the query

?friend :friendOf Charles . (Q₁)

evaluates to a list of people (nodes binding to variable “?friend”) who are friends of (role “:friendOf”) “Charles” (a named node, so a constant).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SIGMOD’16, June 26–July 01, 2016, San Francisco, CA, USA

© 2016 ACM. ISBN 978-1-4503-3531-7/16/06...\$15.00

DOI: <http://dx.doi.org/10.1145/2882903.2882944>

In its latest version, 1.1, SPARQL’s expressiveness is extended with *property paths* [14]. This effectively introduces the concept of *regular path queries* (RPQs)—well studied before the advent of RDF and SPARQL—into the query language. Instead of specifying the path of interest *explicitly* between nodes, one may now specify it *implicitly* via a *regular expression*. For example, the query

?friend :friendOf+ Charles . (Q₂)

evaluates to a list of people who are friends of “Charles”, or friends of people who are friends of “Charles”, and so forth (that is, a *transitive closure* over “:friendOf”).

While SPARQL provides the expressiveness we desire, such queries are more challenging to optimize well. Query Q₁ could be evaluated just by extracting the triples with “r = :friendOf” and “o = Charles”. For even a slightly more complicated query, however, it may not be straightforward to find a *plan* to evaluate it efficiently. Q₂ is more challenging in requiring transitive closure over “:friendOf” with respect to the graph.

Property path evaluation is a tale of two methods: two quite different approaches appear in the literature. For RPQs, the seminal work [18] which introduced the G+ query language shows how to use a *finite state machine* effectively as a plan to guide the *graph walk* for the query’s evaluation. We call this approach FA. Subsequent work on RPQs have followed this idea.

SPARQL with property paths is much more recent. Systems for SPARQL query evaluation have followed a second approach, based essentially on the seminal work of [17]. These extend the *relational algebra* to accommodate the translation of a property path’s regular expression, and then use dynamic programming over the (extended) relational-algebra parse to devise a plan. Added is an “α” operator, which provides transitive closure over a relation to accommodate regular expressions’s Kleene star. Thus, we call this approach α-RA, the relational algebra extended by “α”.

Which approach is better? We shall show that the effective “plan spaces” that result from FA and α-RA are incomparable. Sometimes, for a given query and graph, an FA plan will be the better choice. Other times, an α-RA plan will be. Our goal is to formalize the notion of plan space for both, to be able to choose the best plan. We shall show that a richer plan space can be had that properly subsumes FA and α-RA, and offers more plans existing in neither (“mixed” plans), which sometimes are the best plans.

We have designed a system called WAVEGUIDE with the goal to provide viable cost-based query optimization and

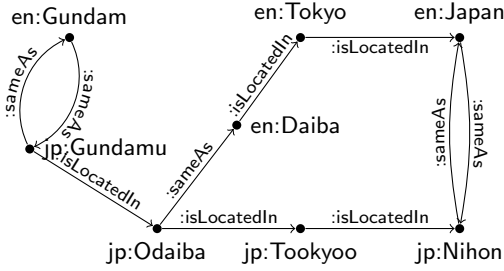


Figure 1: An example graph database.

evaluation for SPARQL over RDF stores that is on par with the state of the art for relational database systems. In this paper, we address the first but critical step of this endeavor, defining a *plan space*—the space of query plans, *wave-plans* (WPs)—for SPARQL queries. We focus on single-path, property-path queries, essentially the RPQ fragment of SPARQL 1.1. We consider a set semantics—the distinct directive in each query—and thus do not consider aggregation.

Our contributions are as follows.

1. *plan space*. (§2—§4)
 - (a) Summarize the state of the art for evaluation of RPQs and SPARQL property paths. (§2)
 - (b) Establish why none suffices. (§3.1)
 - (c) Devise WAVEGUIDE’s place space, and demonstrate that it subsumes the state of the art, and extends beyond it. (§3)
 - (d) Model the cost factors that determine the efficiency of plans, and present the powerful optimizations offered by WAVEGUIDE plans. (§4)
2. *performance study*. (§5)
 - (a) Provide a *microbenchmark* over a pertinent RPQ template with realistic queries over real RDF stores / graphs.
 - (b) Substantiate the optimizations of our approach.
 - (c) Justify the necessity of the richer plan space.

2. BACKGROUND & RELATED WORK

2.1 Path Queries

Regular path queries¹ have been considered since *semi-structured* data models were first introduced [2, 6, 13, 18]. The complexity of RPQs for graph databases particularly has been well studied [4, 7]. In [16], the idea of employing NFAs to guide search for RPQ evaluation appears. In [25], a fixpoint evaluation for property paths is presented.

Regular path queries provide a useful mechanism for querying data in many application domains. For example, consider the knowledge base dataset of the Linked Open Data (LOD) cloud. LOD is a community effort which aims to interlink the structural information available in various datasets on the Web (such as Wikipedia and WordNet), and make it available as a single RDF graph. RPQs prove useful in querying such linked data by providing a convenient declarative mechanism which can be used to answer queries without prior knowledge of the underlying data paths.

¹A *regular path query* Q is a tuple (x, r, y) for which x and y are free variables (that range over nodes) and r is a regular expression.

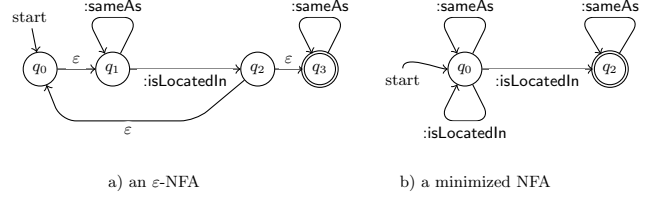


Figure 2: An ϵ -NFA and corres. minimized NFA for Q_3 .

EXAMPLE 1. Consider the part of a LOD graph database as presented in Fig. 1. This represents information about the Gundam robot statue in Odaiba in Tokyo. The data has been integrated from two datasets, identified by the prefixes *en* and *jp*, standing for the English and Japanese Wikipedia, respectively. The entities between these two datasets are interlinked by using OWL ontology terms. Equivalent entities are connected with *owl:sameAs* edges. In this case, the Japanese dataset contains richer spatial information related to the statue than does the English dataset.

Say a user wants to know in which country this Gundam statue is located. Since there are no direct *:isLocatedIn* edges outgoing from *en:Gundam*—as is often the case in linked data—the graph needs to be searched. During the search, equivalent data entities need to be resolved by following *:sameAs* edges. Likewise, a spatial hierarchy needs to be computed by following *:isLocatedIn* edges. This search can be defined by the following SPARQL query pattern:

$$\text{en:Gundam } (:sameAs^*/:isLocatedIn)^+ / :sameAs^* ?place . \quad (Q_3)$$

Query Q_3 computes the spatial hierarchy starting from node *en:Gundam*, using information from both interlinked datasets to resolve equivalent closures.

2.2 Types of Query Plans

The literature on path queries over graphs, as is pertinent to property paths, comes from two distinct sources: evaluation strategies for *regular path queries* (RPQs); and SPARQL platforms extended for version 1.1 to handle *property paths*. We consider each in turn.

FA Plans. Research on RPQs, which well precedes RDF and SPARQL, mostly focused on theoretical aspects, but little on performance issues for evaluating such queries in practice. The seminal work which introduced the G+ query language [18] exploited the natural observation that, where there is a regular expression, there is a *finite automaton* (FA), that is, a *recognizer* for it. In essence, an FA corresponding to the query’s regular expression provides a plan for its evaluation.

EXAMPLE 2. Recall query Q_3 from Ex. 1. As shown in Fig. 2, an automaton construction for this query is a two-step procedure. First, traversing the parse-tree of r bottom up, the ϵ -NFA is built up, from the base case and the inductive rules. Second, the resulting ϵ -NFA is then minimized to an NFA, which typically has smaller size, and hence, is more efficient to process.

The idea of employing a product automaton for RPQ evaluation over graphs has been used in [9, 15, 16, 18, 19, 27] and over data trees (such as XML/XPath) in [5, 22, 23].

α -RA Plans. Work on evaluating property paths—much newer by virtue of the fact that the SPARQL 1.1 standard

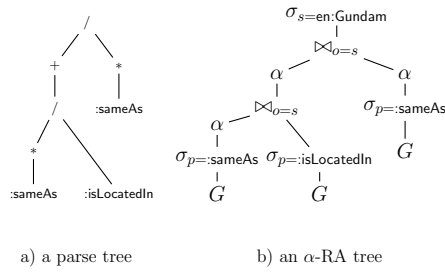


Figure 3: A parse tree and α -RA tree for query $\mathcal{Q}_3$.

is quite recent—has mirrored the dynamic-programming approach behind the algorithm presented in [17]. This can be modeled by the *relational algebra* (RA) extended by an operator α for *transitive closure* (α -RA) [3]. The full power of relational algebra, extended with α , can then be employed to devise an evaluation plan—an α -RA-expression tree—based on the regular expression of the property path. This general approach is found behind many SPARQL platforms as it follows relational techniques well.

The α operator computes the transitive closure of a relation. Let the graph be represented as a relation of triples $G(s, p, o)$. Let $T = \pi_{1,3}G$; thus T consists of pairs of nodes $\langle s, o \rangle$ such that the pair is connected by a directed edge in the graph. Then α applied to T computes the *least fixpoint*:

$$T^+ = T \cup \pi_{1,3}(T^+ \bowtie_{T^+.o=T.s} T) \quad (\mathcal{E}_1)$$

Thus, $\alpha(T)$ results in all pairs of nodes such that, for the nodes of each pair, there *exists* a path between them in graph G . If we were to evaluate the fixpoint by a semi-naïve evaluation, each iteration of evaluation is over paths of length one greater than of the previous iteration. The process stops when no new pairs are added; i.e., the fixpoint has been reached.

Given the relational algebra extended with the α operator, one can evaluate the RPQ $Q = (x, L(r), y)$ over graph $G = (N, E)$ by the algorithm proposed in [17]. This traverses the syntax tree of expression r bottom-up. Let s be the sub-expression of r represented by a given node in a parse tree. The binary relation $R_s \subseteq N \times N$ is computed so that node pair $(u, v) \in R_s$ iff there exists a path from u to v in G matching s .

EXAMPLE 3. Given query $\mathcal{Q}_3$ and the database G from Ex. 1, the corresponding α -RA tree is shown in Fig. 3.

The α -RA-based RPQ evaluation can be directly implemented in most relational databases and relational triple-stores. In [25], a method is proposed that translates RPQs as defined by SPARQL property paths into recursive SQL. A similar approach is used in [11] for the evaluation of provenance-aware RPQs by a relational engine.

3. QUERY PLANS & EVALUATION

Work on property-path evaluation has been remiss in not drawing the connection to RPQs. How do the FA and α -RA approaches compare? Does one subsume the other? Or are they *incomparable*? If so, a combined approach might be superior. A generalized approach might offer new plans that neither FA nor α -RA can produce with superior performance.

Both the FA and α -RA approaches effectively provide evaluation plans for property-path queries. However, the *plan*

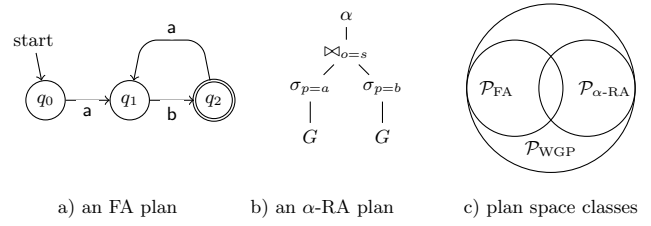


Figure 4: Example plans.

spaces that are implicit in these approaches have not been considered. In FA, choosing a different (but still correct) automaton for the plan might offer a significantly more efficient plan. In systems taking the α -RA approach, planning is done over the α -RA expression tree that results from the property path’s translation, but no planning specific to the semantics of property paths takes place.

In §3.1, we show the approaches of FA and α -RA are, indeed, incomparable. We next describe WAVEGUIDE, which is a generalized approach to query planning and evaluation for SPARQL property-path queries, and which properly subsumes the FA and α -RA approaches. In §3.2, we discuss WAVEGUIDE’s evaluation model. In §3.3, we present WAVEGUIDE’s plans. In §3.4, we show that WAVEGUIDE’s plan space properly subsumes $\text{FA} \cup \alpha\text{-RA}$.

3.1 Limitations

The FA and α -RA approaches each entail a *plan space*; that is, the plans the approach produces over all possible property-path queries. Let $\mathcal{P}_{\text{FA}}$ and $\mathcal{P}_{\alpha\text{-RA}}$ denote the plan spaces for FA and α -RA, respectively. To understand how the approaches are related—for instance, whether one approach subsumes the other, or whether they are incomparable—we consider these plan spaces. The Venn diagram of how they are related is shown in Fig. 4c.²

CLAIM 1. $\mathcal{P}_{\text{FA}}$ and $\mathcal{P}_{\alpha\text{-RA}}$ are incomparable ($\mathcal{P}_{\text{FA}} - \mathcal{P}_{\alpha\text{-RA}} \neq \emptyset$ and $\mathcal{P}_{\alpha\text{-RA}} - \mathcal{P}_{\text{FA}} \neq \emptyset$), but overlap ($\mathcal{P}_{\text{FA}} \cap \mathcal{P}_{\alpha\text{-RA}} \neq \emptyset$).

Of course, we are taking liberties; the place spaces should be over the same *domain* of plans. As we have presented things, however, they are not; we have presented FA plans as automata and α -RA plans as algebraic trees. To prove formally CLAIM 1, we would need to establish an isomorphism between FA and α -RA plans, or have a canonical form for plans to which each plan type could be mapped. This can be done. The formalism for waveguide plans we present in §3.3 would suffice for this mapping. DATALOG, or the *relational algebra* extended by *while loops* (established to be expressively equivalent to DATALOG) [1], would provide an even more universal domain that would suffice.

We can easily establish informally that these spaces are distinct. Consider this *generic* property-path pattern:

$$?x (a/b)^+ ?y. \quad (\mathcal{Q}_4)$$

Here, “a” and “b” are stand-ins for labels. It matches node-pairs that are connected by some path labeled **ab**, **abab**, or **ababab**, and so forth. While quite simple, this already demonstrates the complexities of planning.

²The diagram’s claim that $\mathcal{P}_{\text{WGP}}$ properly subsumes both $\mathcal{P}_{\text{FA}}$ and $\mathcal{P}_{\alpha\text{-RA}}$ is taken up in §3.4.

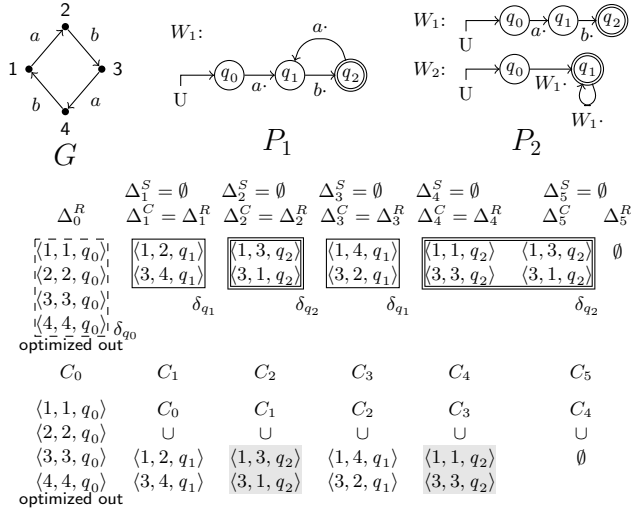


Figure 5: Two waveplans, P_1 & P_2 , over graph G , with an evaluation trace of P_1 .

Consider the FA plan for query $\mathcal{Q}_4$ in Fig. 4a. There is no α -RA plan that could be equivalent to it, however; none would ever evaluate *aba*, *ababa*, and so forth as state q_1 does in the FA plan. α -RA plans cannot compute transitive closure in a *pipelined* fashion as the FA plan is doing; the α operator acts over an entire relation.

Now consider the α -RA plan for query $\mathcal{Q}_4$ in Fig. 4b. There is no FA plan that could be equivalent to it, however; no state transition in its automata can represent the “join” with *ab*. FA plans do not encompass *views* (that is, materialized parts of the query that can be reused), while the α -RA plan does by effectively materializing *ab* to join repeatedly on it.

Meanwhile, FA and α -RA have many plans in common: any query that is restricted to transitive closure over single labels, for example, results in common FA and α -RA plans.

3.2 WAVEGUIDE’S Evaluation

WAVEGUIDE’s evaluation strategy is based on an *iterative search algorithm*, and variations thereof.

Wavefronts. In WAVEGUIDE, we perform efficiently path search while *simultaneously* recognizing the path expressions. WAVEGUIDE’s input is a graph database G and a *waveplan* P_Q which *guides* a number of search *wavefronts* that explore the given graph. We introduce the term *wavefront* to refer to a part of the plan that evaluates breadth-first during the evaluation. This graph exploration, driven by an iterative search procedure, is inspired by the semi-naïve bottom-up strategy used in evaluation of linear recursive expressions based on *fixpoint*, as is done for the α operator for α -RA, described in §2.2.

The key idea is, given a *seed*—a set of nodes in the graph from which this wavefront begins its search—as a start, to expand repeatedly the search wavefronts until no new answers are produced; i.e., we reach fixpoint. Each search wavefront is guided by an *automaton* in the plan, a finite state machine based on an NFA. In Fig. 4a, the plan consists of a single wavefront automaton. We shall see examples below of multi-wavefront plans; e.g., plan P_2 in Fig. 5. This is akin to the FA approach discussed in §2.2. Different, though,

WAVEGUIDESEARCH(G, P_Q)

```

1  $\Delta_0^R \leftarrow \text{seed}(G)$ ;
2  $C_0 \leftarrow \Delta_0^R$ ;
3  $i \leftarrow 0$ ;
4 while  $|\Delta_i^R| \geq 0$  do
5    $\Delta_{i+1}^S \leftarrow \text{seed}(\Delta_i^R)$ ;
6    $\Delta_{i+1}^C \leftarrow \text{crank}(\Delta_{i+1}^S, \Delta_i^R, G, C_i, P_Q)$ ;
7    $\Delta_{i+1}^R \leftarrow \text{reduce}(\Delta_{i+1}^C, \Delta_i^R, C_i)$ ;
8    $C_{i+1} \leftarrow \text{cache}(\Delta_{i+1}^R, C_i)$ ;
9    $i \leftarrow i + 1$ ;
10 done;
11 return extract( $C_i$ );
```

Figure 6: WAVEGUIDE’s evaluation procedure.

from NFAs which are used as recognizers of regular expressions on strings, wavefront automata have features directed to evaluation of regular expressions over graphs.

Expanding a Wavefront.

Each search wavefront has a seed as its initialization. A seed can be either *universal* or *restricted*. A wavefront with a universal seed conducts its search *virtually*³ starting from *every* node in a graph. A wavefront with a restricted seed starts the search from a fixed set of nodes. (A restricted seed is defined by the results of other wavefronts or by constants used in a query.) Graphically, a seed is represented as an incoming edge to starting state q_0 of the wavefront. We use the label “ U ” to denote a universal seed; any other label on this incoming edge denotes a restriction placed on the seed, thus a restricted seed.

The graph exploration (*walk*) is performed by an iterative procedure *guided* by a waveplan, as illustrated in Fig. 6. Consider the example from Fig. 5 with an example graph G and two plans P_1 and P_2 for evaluating query $Q = (x, (ab)^+, y)$ ($\mathcal{Q}_4$). Waveplan P_1 consists of a single wavefront automaton (W_1), which is analogous to the plan that results from the FA method. Fig. 5 also presents the evaluation trace by the procedure in Fig. 6 using plan P_1 over graph G .

During the search, intermediate results are kept in a *cache*, denoted as C_i for iteration i . This is the collection of tuples $\langle u, v, s \rangle$ for which u and v are nodes in G and s is a state in the plan. The newly discovered tuples found in the current iteration are denoted by *delta* Δ_i . We use C_i and Δ_i to eliminate answers already seen in the search. Delta tuples produced by *crank* are denoted Δ^C , and those surviving *reduce* are Δ^R .

To begin, all the universal seeds are initialized (but never materialized!); Δ_0^R is assigned the set of $\langle u, u, q_0 \rangle$ for all $u \in N$. Note that q_0 is the starting state for all wavefronts with universal seeds. We then iterate, performing four operations per iteration:

1. **seed.** Determine the set of nodes from which to search.
2. **crank.** Perform a step of the fixpoint operation, the breadth-first search.
3. **reduce.** Eliminate node pairs seen before.
4. **cache.** Update the set of pairs of nodes seen already in the evaluation.

³Effectively all nodes in a graph are never materialized due to *first-hop* optimization which restricts the starting nodes according to the transition labels originating from q_0 .

The iteration continues until fixpoint is reached. Thus, this is a guided semi-naïve evaluation [1].

The **seed** step populates the restricted seeds, according to their respective seed conditions. The **crank** step transitions from the previous delta to the current, $\Delta_i^R \rightarrow \Delta_{i+1}^C$. For each node v in $\langle u, v, s \rangle \in \Delta_i^R$, for edge $\langle v, a, w \rangle \in G$ and automaton transition $\langle s, a, t \rangle \in W_1$, $\langle u, w, t \rangle$ is added to Δ_{i+1}^C . Thus **crank** advances the search *simultaneously* in the graph *and* in the automaton.

To prevent unbounded computation over cyclic graphs, the delta is *reduced*: Δ_{i+1}^C is checked against both the previous delta Δ_i^R and the cache C_i ; tuples that are seen in either Δ_i^R or C_i are removed to produce Δ_{i+1}^R . Lastly, the cache is updated C_{i+1} by adding the tuples in the reduced delta Δ_{i+1}^R to it (C_i). The iteration halts once Δ^R is empty.

It can be established by structural induction that, for any tuple $\langle u, v, s \rangle$ in the cache (C) such that s is an accepting state, the pair of nodes $\langle u, v \rangle$ must have a path between them in the graph that conforms to r . Thus, WAVEGUIDE produces the correct results. The answer set can be then *extracted* from the cache by selecting the tuples $\langle u, v, s \rangle$ for all accepting states s of automaton W_1 .

EXAMPLE 4. Consider the evaluation trace presented on Fig. 5. First, universal seeds in Δ_0^R are optimized out by the first-hop optimization which builds Δ_1^C directly by selecting the tuples from G which match the label (a) of the first transition $\langle q_0, a, q_1 \rangle$ in P_1 . The search proceeds by iterating **crank**, **reduce** and **cache** to produce Δ_i^C , Δ_i^R and C_i at each iteration i . The search stops at the 5th iteration when reduced delta Δ_5^R is empty. Those cache tuples in accepting state q_2 (shaded) at the end are then extracted as the answer set.

3.3 Waveplans

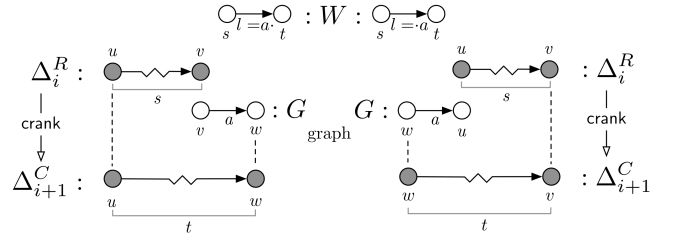
Guiding a Wavefront. The construction of the NFA forces an order to the query evaluation. A “wrong” choice of NFA can lead to an inefficient evaluation plan. In WAVEGUIDE, we aim to minimize the search space explored by considering the possible orders of graph exploration by search wavefronts. To achieve this, we use wavefront automata which can use transitions that expand the wavefront in the direction opposite to the direction of the edges of the graph.

Consider a graph transition $\langle s, l, t \rangle$ in a wavefront as illustrated in Fig. 7a. Edge label l has a general form $\cdot a$ or $a \cdot$, where a is an edge label in G and the position of a dot ‘ $\cdot$ ’ specifies a direction of a search wavefront and denotes prepend ($\cdot a$) or append ($a \cdot$) transition. A prepend transition expands in the opposite direction to the edges in the graph. On the other hand, an append transition guides a wavefront that expands in the same direction as the edges in the graph.

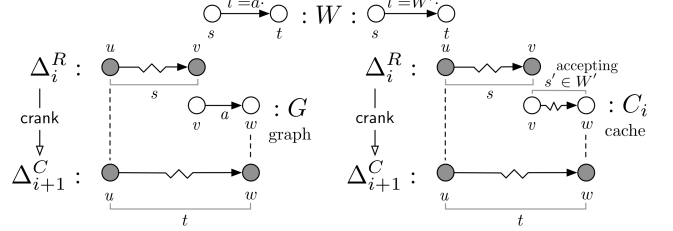
Hence, wavefronts enable automaton transitions that explore the graph in a specified direction. This allows a wavefront to initiate evaluation from any label in the given regular expression and iteratively expand by appending or prepending path labels. This gives us the power to explore all different expansion orders of a single wavefront.

Wavefront Interaction. Often, the search space is constrained even further if several wavefronts are employed in the evaluation, each evaluating parts⁴ of a given regular ex-

⁴This decomposition of a regular expression can be determined by graph statistics such as cardinalities of its labels.



(a) Prepending vs. appending by expanding the wavefront using tuples from graph G .



(b) Expanding the wavefront by appending and using the tuples from G (over the graph) vs. from search cache C (over the view).

Figure 7: Types of transitions used in a wavefront.

pression. A WAVEGUIDE plan enables this by defining a number of automata, one for each search wavefront.

In addition to transitions over graph edge labels, waveplans allow transitions over *path views*, by using cached result sets produced by other wavefronts. Consider Fig. 7b and automaton transition $\langle s, l, t \rangle$. If l is an edge label a in G , then this *graph* transition expands the wavefront by using the tuples from *graph* G . Otherwise, if l is a wavefront W' then this *view* transition expands the wavefront by employing the tuples produced by the corresponding wavefront (W').

These new types of transitions offer powerful choices in WPs for guiding the search. The search can have *multiple* wavefronts originating from different starting points *and* expanding in different directions. Further, each wavefront can employ the cache through transitions over views to avoid recomputation. (This is also known as *memoization*.)

EXAMPLE 5. Consider the wavefront search in Fig. 5 for a query Q with regular expression $r = (ab)^+ (\mathcal{Q}_4)$. P_1 is a basic WP embodying an NFA that recognizes r . From P_1 , we can design a more efficient WP, P_2 : first, compute (ab) with wavefront W_1 ; then use a loop-back path-view transition to compute the closure $(ab)^+$ (with wavefront W_2). It can be shown that P_2 explores a smaller search space than P_1 .

3.4 WAVEGUIDE’s Plan Space

We claim that the space of waveguide plans subsumes that of the FA and α -RA approaches, as the Venn diagram in Fig. 4 shows (and with the caveats as discussed in §3.1).

CLAIM 2. $\mathcal{P}_{WP}$ properly subsumes the union of $\mathcal{P}_{FA}$ and $\mathcal{P}_{\alpha-RA}$ ($\mathcal{P}_{WP} \supsetneq \mathcal{P}_{FA} \cup \mathcal{P}_{\alpha-RA}$).

That $\mathcal{P}_{WP}$ subsumes each of $\mathcal{P}_{FA}$ and $\mathcal{P}_{\alpha-RA}$ is straightforward; we devised WP so that we could express both FA- and α -RA- type plans. WP extends the FA model. WP encompasses α -RA by the addition of *path views*; what the α operator offers, transitive closure over an arbitrary relation, can be accomplished by view-labeled transitions.

$\mathcal{P}_{WP}$ properly subsumes the union of $\mathcal{P}_{FA}$ and $\mathcal{P}_{\alpha\text{-RA}}$ because there is a waveguide plan that corresponds to neither an FA plan nor an $\alpha\text{-RA}$ plan. We have demonstrated that in the discussions above: any WP with multiple wavefronts and some wavefront with a long loop-back is such a plan; FA plans are essentially single wavefront by the FA model, and pipelined loop-backs are outside the scope of $\alpha\text{-RA}$. Likewise, any WP, even single wavefront, that is “mixed”, that combines views and long loop-backs, corresponds to no FA plan and to no $\alpha\text{-RA}$ plan. (In Fig. 11 below, P_2 with partial loop-caching is such a plan.) These very types of waveguide plans that FA and $\alpha\text{-RA}$ miss can be the most efficient plans.

In §4, we explain why this rich plan space is relevant. In §5, we compare plans for real queries over real graph data to establish that this is true in practice, as well.

4. PLAN COSTS

We present a *cost framework* for WAVEGUIDE search, *search cost factors* that can magnify the cost (properties of the graph and of resulting pre-paths computed during evaluation), and *optimization methods* that are enabled by WPs which address the search factors, in turn.

4.1 Cost Framework

Recall the three steps in Fig. 6 of the search iteration: crank, reduce, and cache. Assume that the search completes in n iterations. The cost of **crank**, C_{crank} , corresponds to the total number of *edge walks* performed. This *search size* is the sum of sizes of the deltas (Δ^C ’s). The cost of **reduce**, has two components: duplicate removal within a delta (C_{reduce}^Δ) and for the delta against the search cache (C_{reduce}^C). The cost of **cache**, C_{cache} , is associated with search cache maintenance procedures (e.g., indexing). The functions f_{1-4} are placeholders that weigh the costs:

Step	Generalized Cost
C_{crank}	$\sum_{i=0}^n f_1(\Delta_i^R , G , C_i)$
C_{reduce}^Δ	$\sum_{i=1}^n f_2(\Delta_i^C)$
C_{reduce}^C	$\sum_{i=1}^n f_3(\Delta_i^C , C_i)$
C_{cache}	$\sum_{i=1}^n f_4(\Delta_i^R , C_i)$

The cost functions f_{1-4} above are monotone over their parameters; these simply abstract the actual costs as based upon the underlying implementation of WAVEGUIDE’s data structures and algorithms. For example, in a relational-based system, assuming tree indexes on C_i and G , functions f_{1-4} can be approximated as:

	Example Implementation	Complexity
f_1	Index-nested loop joins: $\Delta_i^R \bowtie_{P_Q} G$ and $\Delta_i^R \bowtie_{P_Q} C_i$.	$O(\Delta_i^R \cdot (\log G + \log C_i))$
f_2	Sort Δ_i^C , remove duplicates.	$O(\Delta_i^C \cdot \log \Delta_i^C)$
f_3	Scan Δ_i^C , probe C_i ’s index.	$O(\Delta_i^C \cdot \log C_i)$
f_4	Scan Δ_i^R , insert into C_i ’s index.	$O(\Delta_i^R \cdot \log C_i)$

In this scenario, crank is implemented as joins of Δ_i^R with G (for graph transitions) and C_i (for view transitions). The predicates of these joins are dictated by the WP P_Q . Since usually $\Delta_i^C \ll C$, reduce^Δ typically occurs in-memory. On the other hand, reduce^C is more expensive since it requires probing C ’s index. Finally, **cache** involves updating C ’s index with new Δ_i^R ’s tuples.

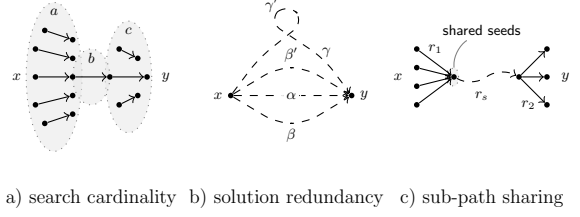


Figure 8: Types of search cost factors.

4.2 Search Cost Factors

Properties of the graph and of the WP chosen will determine the evaluation cost.

Search Sizes. The wavefronts that we choose for the search determine the intermediate Δ_i (pairs of nodes connected by valid pre-paths) that we collect in each iteration i . Just as with different join orders in relational query evaluation, different wavefronts will result in different cardinalities of Δ_i . These intermediate cardinalities can vary widely from plan to plan and affect all four costs f_{1-4} .

Solution Redundancy. After much deliberation, the W3C has adopted a *non-counting* semantics for SPARQL property-path queries. Each node pair appears only once in the answer, even when there are several paths between the node pair satisfying the given regular expression.

Answer-path redundancy arises from two sources. First, in dense graphs, solutions are re-discovered by following conforming, yet different paths. Second, nodes are revisited by following cycles in the graph. Thus, the same answer pair may be discovered repeatedly during evaluation, which increases both $|\Delta_i|$ and $|C_i|$. It is critical to detect such duplicate solutions early in order to keep the costs f_{1-4} low.

Sub-path Redundancy. In *solution redundancy*, an answer pair could have multiple paths justifying it. Likewise, the paths justifying multiple answer pairs may *share* significant segments (*sub-paths*) in common. This arises, for instance, in dense graphs and with hierarchical structures (e.g., *isA* and *locatedIn* edge labels). Consider a query “?p :locatedIn+ Canada”. Every person located in the neighborhood of the Annex in the city of Toronto qualifies, since the Annex is located in Toronto which is located in Ontario which is located in Canada. The sub-path “Annex :locatedIn+ Canada” is shared by the answer path for each Annex resident. Because we keep only node-pairs (plus state) in the search deltas, and not explicitly the paths themselves,⁵ we may walk these sub-paths many times, recomputing “Annex :locatedIn+ Canada” for each Annex resident, thus unnecessarily increasing both $|\Delta_i|$ and $|C|$.

4.3 Plan Optimizations

We consider WP-optimization methods in relation to the search cost factors above.

Choice of Wavefronts. The direction in which we follow edges, and where we start in the graph, with respect to the regular expression will result in different $|\Delta_i|$ we encounter during the search. Our choice of automata in the WP dictates the wavefront(s). For example, consider query $Q = (x, (abc), y)$ and a fragment of a graph shown in Fig. 8a.

⁵Note this design choice in our evaluation strategy is critical for good performance due to solution redundancy!

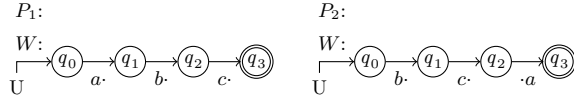


Figure 9: Choosing the wavefronts.

Since labels a , b and c have different frequencies, different wavefronts will have different search sizes. Consider two plans P_1 and P_2 that evaluate Q shown in Fig. 9. P_1 has a single wavefront that explores the graph starting from a , appending b and then c . On the other hand, P_2 has a wavefront that starts from the low frequency label b , appends c and then prepends a . Observe that, in this scenario, P_2 results in fewer edge walks than P_1 .

To reduce overall search size, we need to choose wavefronts that result in fewer edge walks. Wavefronts can be costed to estimate their search sizes based on statistics about the graph, such as 1-gram and 2-gram label frequencies. (Such graph statistics can be computed offline for this purpose.)

Reduce. WAVEGUIDE's evaluation strategy is designed to counter solution redundancy. As shown in Fig. 8b, we consider several types of redundant solutions based on a path which was followed to obtain each solution. Path α is the shortest path. Paths β and β' are of the same length, but go through different nodes. Finally, path γ' shares some nodes with path γ , but it is longer due to a cycle.

In WAVEGUIDE, redundancy of *candidate* solutions is addressed by removal of duplicates against both C and Δ by the **reduce** operation. Assuming a BFS search strategy, duplicate solutions obtained by following paths of the same length (β , β') are removed within a Δ . On the other hand, duplicates obtained by following paths of different lengths (α , β) are removed when Δ is compared against C . This also includes paths with cycles such as γ and γ' as they also have different lengths.

As a further optimization, once a solution seed-target pair has been discovered, *first-path pruning* (fpp) removes the *seed* from further expansion by the search wavefronts. In our example, once path α has been discovered and solution (x, y) has been obtained, all longer paths (β , β' , γ , γ') are never even materialized.

Threading. To counter *sub-path redundancy* requires us to decompose a query into sub-queries. We call this decomposition *threading*, and our WPs accommodate this.

Consider query $Q = (x, (r_1 r_s r_2), y)$ where sub-path r_s is shared among many solutions as shown in Fig. 8c. This query can be threaded as follows. First, pre-path r_1 is computed by wavefront W_{r_1} . Then, the portion of the regular expression that will result in sub-paths that will be shared by many answer paths can be computed by a separate wavefront $W_{r_1:r_s}$. Here, W_{r_1} seeds wavefront $W_{r_1:r_s}$ which computes a shared path r_s for each of the partial solutions produced by W_{r_1} . Finally, the complete path is pieced together by wavefront W_{join} . Sub-path sharing can be predicted by graph statistics to indicate when threading is useful.

Partial Caching. Delta results are cached during evaluation as we need to check against C for redundantly computed pairs. For large intermediate $|\Delta_i|$, this can be a significant cost. However, some of this cost can be eliminated. In particular, not every state in the WP's automata needs to have its node-pairs cached. Caching is only needed when

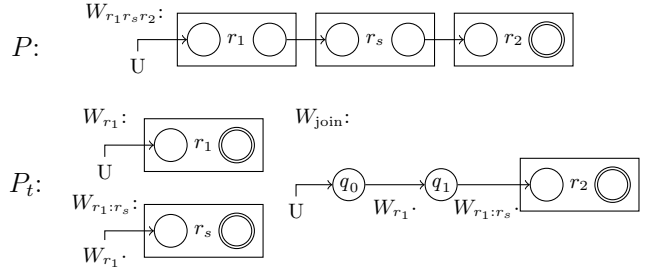


Figure 10: Threading a shared sub-path.

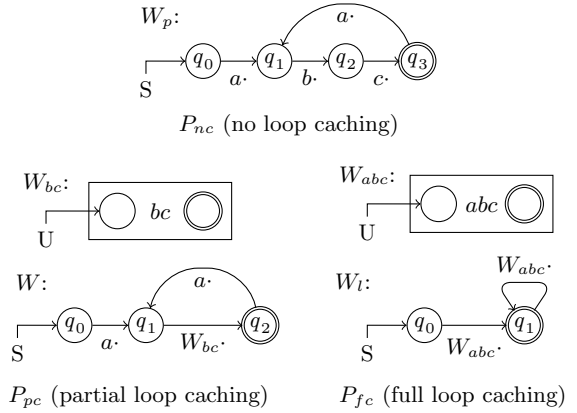


Figure 11: Types of loop caching.

unbounded redundancy is possible, due to cycles in the wavefront automata or in the graph. States without cycles need not be cached.

Loop Caching. Transitions over views in wavefront automata allows us to cache and re-use some of the intermediate node pairs we encounter during the search. Such named result sets are useful in reducing unnecessary re-computation by employing an optimization we call *loop caching*.

In transitive query $Q = (x, (r)^+, y)$, the expression r is evaluated repeatedly until no new solutions are found. Loop caching rewrites an evaluation plan so that the base r is cached either fully or partially to speed up the transitive evaluation of $(r)^+$.

Consider three plans P_{nc} , P_{pc} and P_{fc} for query $Q = (x, (abc)^+, y)$ shown in Fig. 11. Plan P_{nc} has no loop caching as it evaluates full expression (abc) in a pipeline with W_p . Plan P_{pc} uses a separate wavefront to evaluate (bc) first, then these results are used in a loop to evaluate transitive $(abc)^+$. Finally, plan P_{fc} caches the full base expression (abc) in W_{abc} , which is then used in evaluation of a transitive expression in a loop by W_l .

4.4 Cost Analysis

Consider deltas Δ_i^C , Δ_i^R and cache C_i for all iterations in the evaluation of WP P_Q over graph G . These determine the overall cost of P_Q according to the framework in §4.1.

We define a *logical* delta δ_s over state s to be a set of delta tuples $(u, v, s) \in \delta_s$ such that $\delta_s \subseteq \bigcup_{i=0}^n \Delta_i$ (assuming n total iterations of the search). In other words, from all *physical* deltas Δ_i we encountered during the search, we *select* delta tuples which occur in state s . Logical delta δ_S over set of states S is a union of logical deltas over all states

in S , i.e. $\bigcup_{s \in S} \delta_s$. Similarly, δ_W over wavefront W is a union $\bigcup_{s \in W} \delta_s$. For example, consider a breakdown of Δ into δ for an execution of P_1 as shown on Fig. 5. Here, for each state $q \in P_1$, δ_q is shown by encasing the Δ tuples in a dashed, single and double rectangles for δ_{q_0} , δ_{q_1} and δ_{q_2} , respectively.

Cost of Threading. Given a plan P with a single wavefront $W_{r_1 r_s r_2}$ which computes a regular expression of a form $r = r_1/r_s/r_2$, threading rewrites it into a plan P_t with three wavefronts W_{r_1} , $W_{r_1:r_s}$ and W_{join} as described in Section 4.3. We partition states in $W_{r_1 r_s r_2}$ into three subsets: S_{r_1} , S_{r_s} and S_{r_2} which are responsible for evaluation of expressions r_1 , $r_1 r_s$ and $r_1 r_s r_2$, respectively.

To analyze the cost of threading, we partition edge walks into logical deltas for both P and P_t as follows:

$$P : \sum_{i=0}^n |\Delta_i^C| = |\delta_{W_{r_1 r_s r_2}}^C| = |\delta_{S_{r_1}}^C| + |\delta_{S_{r_s}}^C| + |\delta_{S_{r_2}}^C| \quad (1)$$

$$P_t : \sum_{i=0}^{n_t} |\Delta_i^{''C}| = |\delta_{W_{r_1}}^C| + |\delta_{W_{r_1:r_s}}^C| + |\delta_{W_{\text{join}}}^C| \quad (2)$$

From construction (Fig.10), we have:

$$|\delta_{W_{r_1}}^C| = |\delta_{S_{r_1}}^C| \quad (3)$$

$$|\delta_{W_{\text{join}}}^C| = |\delta_{q_0}^C| + |\delta_{q_1}^C| + |\delta_{q_2}^C| \quad (4)$$

We define a *multiplicity* ratio of an expression r_1 in graph G , which is computed by analyzing the paths in G :

$$\mathcal{M}(G, r_1) = \frac{|S_s|}{|S_o|},$$

where S_s and S_o is a set of subjects and objects, respectively, connected in G with paths conforming to r_1 . Then, $\mathcal{M}(G, r_1) > 1$, would indicate that, on average, there are many subjects connected to a single object in G , while $\mathcal{M}(G, r_1) < 1$ would indicate that the opposite is true. The greater $\mathcal{M}$ is, the more subjects are connected to the same object by path r_1 , and, hence, more subjects share a path r_s which originates from this object. Hence, we can estimate:

$$|\delta_{S_{r_s}}^C| = \mathcal{M}(G, r_1) \cdot |\delta_{W_{r_1:r_s}}^C| \quad (5)$$

We define a *gain* $\mathcal{G}_t$ introduced by threading in P as difference in a number of edge walks performed in P and P_t . From (1-5), this gain is:

$$\mathcal{G}_t = \underbrace{(\mathcal{M}(G, r_1) - 1) \cdot |\delta_{W_{r_1:r_s}}^C|}_{\text{A: savings due to threading}} - \underbrace{(|\delta_{q_0}^C| + |\delta_{q_1}^C|)}_{\text{B: extra join}}$$

If a shared sub-path r_s is accurately identified, then the total reduction of number of edge walks in P_t (**A**) is sufficiently large to offset the cost of the extra join (**B**), i.e. $\mathcal{G}_t > 0$.

To maximize **A**, we need to maximize both $\mathcal{M}(G, r_1)$ and $|\delta_{W_{r_1:r_s}}^C|$. For the latter, one useful metric to consider is an average length $\mathcal{L}(G, r_s)$ of a path which conforms to r_s in G . A longer shared sub-path r_s can increase the cardinality $|\delta_{W_{r_1:r_s}}^C|$; hence, more potential savings in edge walks can be realized by a threading split on r_s .

Thus, given $\mathcal{M}$ and $\mathcal{L}$ for sub-expressions of r in G , the identification of an efficient threading split $r = r_1/r_s/r_2$ becomes an $(\mathcal{M}, \mathcal{L})$ maximization problem.

Cost of Loop Caching. Given a plan with a single wavefront which computes closure $(r)+$ of a regular expression r ,

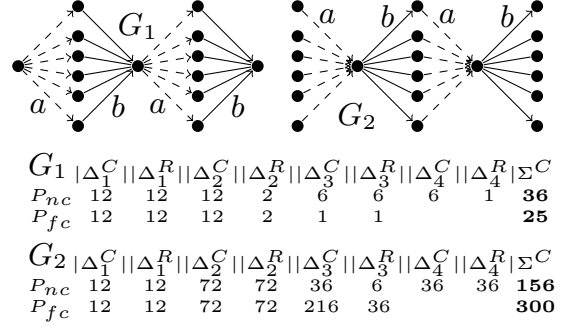


Figure 12: Lensing.

loop caching rewrites it into a plan in which parts of r are pre-computed, cached, and then used in an iterative evaluation of the closure. For example, consider the differences in evaluation of query $Q = (x, (abc)+, y)$ with plans P_{nc} , P_{pc} and P_{fc} shown in Fig. 11. P_{nc} defines a single wavefront, which, due to absence of transitions over views, can be executed pipelined. On the other hand, P_{pc} and P_{fc} first compute (bc) and (abc) , respectively, in separate wavefronts, the results of which are used in a wavefront which computes the final closure.

As previously, to analyze the cost of loop caching, we partition edge walks into logical deltas for both P_{nc} and P_{fc} :

$$P_{nc} : \sum_{i=0}^{n_{nc}} |\Delta_i^C| = |\delta_{q_0 \in W_p}^C| + \underbrace{|\delta_{q_1 \in W_p}^C| + |\delta_{q_2 \in W_p}^C| + |\delta_{q_3 \in W_p}^C|}_{\mathbf{C}}$$

$$P_{fc} : \sum_{i=0}^{n_{fc}} |\Delta_i^{''C}| = \underbrace{|\delta_{W_{abc}}^C|}_{\mathbf{D}} + |\delta_{q_0 \in W_l}^C| + \underbrace{|\delta_{q_1 \in W_l}^C|}_{\mathbf{E}}$$

From construction (Fig.11), we have:

$$|\delta_{q_0 \in W_p}^C| = |\delta_{q_0 \in W_l}^C|$$

It follows that the gain $\mathcal{G}_{lc}$ of loop caching is the difference in a number of edge walks in P_{nc} and P_{fc} :

$$\mathcal{G}_{lc} = \mathbf{C} - (\mathbf{D} + \mathbf{E})$$

In order for cached plan P_{fc} to be faster, the sum of edge walks associated with computation of base (abc) paths (**D**) and looped evaluation of $(abc)+$ paths (**E**) should be less than the sum of edge walks associated with evaluation of $(a), (ab), (abc), (abca), \dots$ paths (**C**) in pipelined plan P_{nc} .

Interestingly, values of terms **C** and **E** can significantly differ depending on the general shape of the graph. For example, consider two basic graphs G_1 and G_2 as presented in Fig. 12. Both G_1 and G_2 have the same frequencies of labels a and b , but are different in terms of their *shape*. G_1 exhibits *lensing* with focal points on concatenations b/a , while G_2 has lensing in a/b . Intermediate cardinalities of Δ^C (number of edge walks) of the WAVEGUIDE search are presented for plans with (P_{fc}) and without (P_{nc}) loop caching. Observe that loop caching optimization is beneficial for search in G_1 with 30% fewer edge walks. On the other hand, loop caching performs worse in G_2 with 92% more edge walks. This can be explained by analyzing the edge walks and pruned tuples during the concatenation sequence $(\dots/a/b)$ which is

⁶We omit the comparison with P_{pc} as it follows the same pattern.

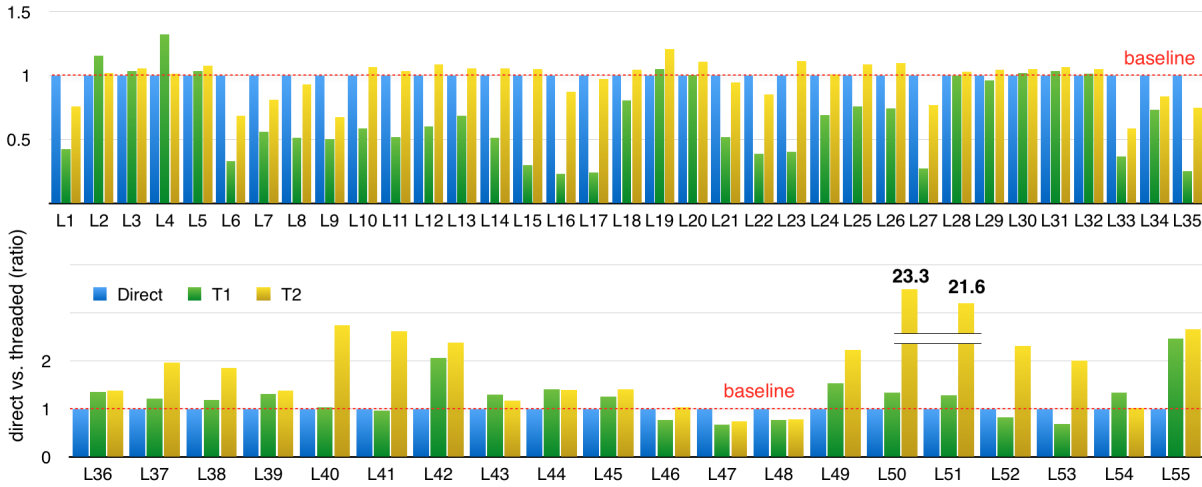


Figure 13: Benchmarking Threading.

performed in P_{nc} , but not in P_{fc} . In G_1 , $(\dots/a/b)$ computes a large number of intermediate tuples, most of which are later pruned due to the focal point in b/a . Meanwhile, in G_2 , $(\dots/a/b)$ first prunes many tuples due to the focal point in a/b , hence reducing the total number of edge walks performed later in the search.

Lastly, we consider queries with constants. In pipelined plan P_{nc} , this constant can be pushed to seed condition S of its wavefront, therefore reducing term **C**. On the other hand, plans P_{pc} and P_{fc} allow at most partial constant pushdown, since cached relations must be computed (term **D**) with universal seed to ensure completeness of the final closure.

5. PERFORMANCE STUDY

5.1 The Experimental Setup

The WAVEGUIDE Prototype. We have prototyped a system [26] that implements the methodology from §3 in order to benchmark waveguide plans to study their performance. In this WAVEGUIDE system, resource-intensive tasks are delegated to POSTGRESQL via SQL and *procedural* SQL routines.

Fig. 17 (in the Appendix) shows the architecture. It consists of two layers: *application* and *RDBMS*. The application layer provides a user front-end, preprocessing the graph data, parsing user queries, generating WPs, and visualizing key steps during the search. The RDBMS layer provides postprocessing of the graph data and performing the iterative WAVEGUIDE graph search for the given WP.

Our implementation of WAVEGUIDE’s operations matches the example implementations (f_1 – f_4) given in §4.1.

Methodology. We test our implementation of WAVEGUIDE by running a collection of realistic path queries over real-world datasets YAGO2s [24] and DBPedia [10]. The datasets were preprocessed by removing invalid and duplicate triples. After preprocessing, YAGO2s had 242M triples and DBPedia had 463M triples, with 104 and 65K distinct predicates, respectively. Sufficiently large number of predicates (> 100) makes these datasets well suited for benchmarking of path queries.

At the time of this paper, we could not find any available benchmarks for SPARQL property-path queries. We there-

fore generate path queries based on data patterns we identified in real-world graphs. The goal of these experiments is to verify the gains offered by WAVEGUIDE optimizations, and show that they correspond to our cost framework (§4.1) and analysis (§4.4).

5.2 The Optimizations

Threading. We benchmark the threading optimization by executing a query of the following template:

$$?x \ p/p_1+/p_2+ \ ?y \quad (\mathcal{Q}_5)$$

over the YAGO2s dataset, with “ p ”, “ p_1 ” and “ p_2 ” as variable predicates. We chose this template because it contains the concatenation of two transitive closures, which makes it difficult to predict the average length of the paths in the answer.

We group p candidates in two sets: the first (queries **L1**–**L35**) having an $\mathcal{M}$ value greater than 10; and the second (queries **L36**–**L55**) having an $\mathcal{M}$ value less than 1. Each of the queries is executed with three different plans: D , a direct evaluation with a single wavefront with no threading; T_1 threads on shared path (p_1+/p_2+) ; and T_2 threads on (p_2+) to illustrate the effect of potentially shorter $\mathcal{L}$ than in T_1 . Given p , predicates p_1 and p_2 are chosen such that the running time of $\mathcal{Q}_5$ is between 0.5 and 5 minutes.

The relative running times for queries **L1**–**L55** executed with plans D (the baseline), T_1 , and T_2 are presented in Fig. 13. As anticipated, the evaluation of queries in the first group is significantly (up to 75%) faster threaded than direct, and with T_1 being faster than T_2 . This can be attributed to the fact that the length of the shared path $\mathcal{L}$ is shorter in T_2 due to a “later” threading split in the query expression. Also as anticipated, queries in the second group show bad results for T_1 . Indeed, picking a predicate with $\mathcal{M} < 1$ for a threading split will generally be bad due to few shared paths. Further, some queries (**L50**, **L51**) are evaluated significantly more slowly (up to 23 times!) when threaded in T_2 . This illustrates the importance of choosing the correct threading split.

Loop Caching. We benchmark the loop-caching optimization by executing a collection of queries of the simple template $Q_{(ab)+} = (x, (ab)+, y)$ ($\mathcal{Q}_4$) with two WPs P_{nc} and

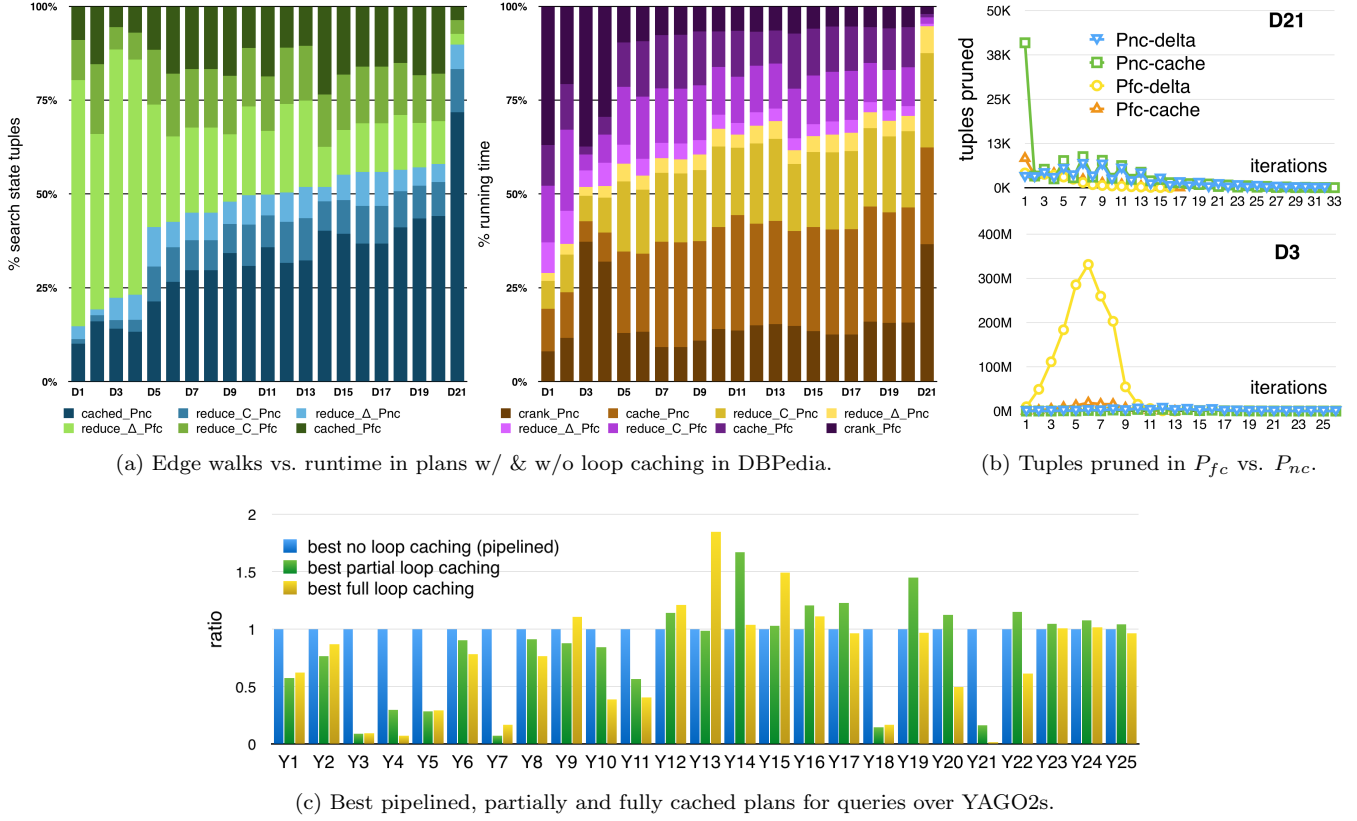


Figure 14: Benchmarking Loop Caching.

P_{fc} , which specify executions of $Q_{(ab)+}$ with no loop caching and with full loop caching, respectively.

Values for a and b were chosen by iterative pruning of predicates appearing in the DBPedia dataset. First, we excluded predicates with very high (more than 25M) and low (less than 75K) cardinalities, to keep query running times reasonable. Then, we ran query $Q_{abab} = (x, (abab), y)$ and recorded those (a, b) predicate pairs for which the result of Q_{abab} was not empty. DBPedia had 1171 such pairs, which indicates a high number of $(ab)+$ paths in this dataset. For each of these pairs, we ran the full closure query $Q_{(ab)+}$ to obtain its *expansion ratio*,

$$r_{\text{exp}} = \frac{|Q_{(ab)+}|}{|Q_{ab}|} \quad (\mathcal{E}_2)$$

where $|Q|$ denotes the cardinality of a query result.

Recall that both P_{nc} and P_{fc} initially evaluate (ab) paths in the same way, while the rest of the closure $(ab)+$ is computed differently. Hence, in order to show the differences between these plans, we chose predicate pairs with $r_{\text{exp}} > 2$, so that the computation of the rest of the closure constitutes at least 50% of the plan execution time. We identified 21 such queries by analyzing graph patterns in DBPedia.

We evaluated each one of these queries with P_{nc} and P_{fc} plans and recorded the *running time*, *edge walks* and *pruning statistics*. Due to widely varying absolute values for these statistics across queries, we present their relative percentage breakdowns in Fig. 14a, as follows. Each query is represented by a multi-colored bar, which shows the percentage breakdown of statistics values between P_{nc} and P_{fc} executions. In this way, we present edge walks (in the left chart)

and running-time execution (in the right chart). We enumerate the queries from **D1** to **D21** by sorting ascending with respect to the percentage of edge walks performed in the P_{nc} execution relative to the P_{fc} execution. Hence, in query **D1**, the P_{nc} execution resulted in significantly fewer edge walks relative to P_{fc} execution, with the opposite true for query **D21**. For edge walks, we perform a further breakdown for each query of the total number of edge walks into the number of tuples which were cached, were reduced against the cache, or were reduced against the delta. This breakdown is represented by different shades of the color associated with P_{nc} or P_{fc} executions, respectively. Similarly, for each query, we break down its running time into total times of **crank**, **cache**, **reduce^C** and **reduce^Δ** steps to illustrate the costs of each in relation to the framework presented in §4.1.

Our first observation is that, in general, the loop caching optimization can both significantly increase *or* decrease the total number of edge walks performed by the search. In our benchmark, loop caching resulted in fewer edge walks in 10 queries, with almost an order of magnitude reduction, in the best case. On the other hand, in 11 queries, loop caching resulted in more edge walks, with a more than 5X increase, in the worst case.

Our second observation is that the real cost of WAVEGUIDE search corresponds to the approximations of costs f_1 – f_4 given in §4.1. Consider queries **D1**–**D11** in which loop caching resulted in a higher total number of edge walks. However, the majority of these queries still ran faster than their pipelined counterparts due to the fact that the bulk of edge walks produced duplicate tuples, which were removed by **reduce^Δ**. Indeed, such in-memory removals are

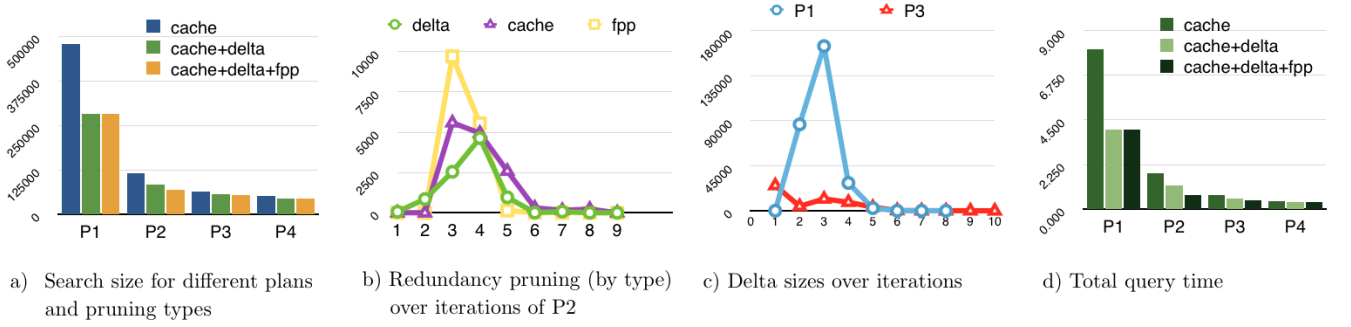


Figure 15: Effect of plans on query evaluation.

inexpensive according to f_2 . Further, we observe that expensive (f_1, f_3 and f_4) disk-based operations **crank**, **reduce^C** and **cache** which require probing or maintaining a tree index contribute significantly to the overall cost.

Finally, we study the effect of lensing by analyzing the degree of **delta** and **cache** pruning. Fig. 14b plots pruning over iterations for the queries which exhibit lensing: **D3** and **D21**. Query **D3** has $\mathcal{M}(G, a) = 10.58$ and $\mathcal{M}(G, b) = 0.33$, which suggests lensing with focal point on the concatenation a/b . As discussed in §4.4, this can significantly increase amount of pruning for loop caching, which is indeed what we observe. On the other hand, **D21** has $\mathcal{M}(G, a) = 0.07$ and $\mathcal{M}(G, b) = 5.34$, which suggests lensing with focal point on the concatenation b/a . This lensing benefits loop caching by decreasing the amount of pruning over iterations, which is what we observe.

Partial Loop Caching. In previous section we have shown experimentally that plans from both FA and α -RA spaces need to be considered in query evaluation. In this section, we aim to show that plans which are not in FA nor in α -RA often also require consideration.

We design a collection of queries over YAGO2s dataset which are based on the template:

$$?x (a/b/c) + ?y \quad (\mathcal{Q}_6)$$

We pick values for a , b and c by iterative pruning of predicates appearing in the YAGO2s dataset similar to the process discussed in §5.2. As a result, we came up with 25 queries **Y1–Y25** which follow template $\mathcal{Q}_6$.

The results of the benchmark are presented on Fig. 14c. We group plans into three categories: no loop caching (pipelined), partial loop caching and full loop caching. For each category we pick a best plan as judged by a total number of edge walks performed. Due to wide differences in absolute values, we present the best pipelined plan as a baseline and present the other two groups in relation to it.

As expected, pipelined plans are often not the best choice in evaluation of open-ended queries like $\mathcal{Q}_6$. Here, pipelined plans win for five out of 25 queries. This is explained by the strict evaluation order imposed by such plans. For a query like $\mathcal{Q}_6$, there are only two legal pipelined plans, direct and inverse pipelines. Hence, often these plans lose to other plans which allow more flexibility in evaluation order. Fully loop cached plans, which allow the most flexibility in evaluation order (we generate 20 such plans for $\mathcal{Q}_6$), are fastest in 11 queries out of 25, sometimes by an order of magnitude. Finally, in nine queries out of 25, partially loop cached plans are the fastest, sometimes by a large margin.

Hence, these plans, which are exclusive to WAVEGUIDE, need to be considered as well.

Combined Optimizations. We illustrate the impact of combining WAVEGUIDE optimizations over the example query

$$?p :marriedTo/:diedIn/:locatedIn+/:dealsWith+ USA \quad (\mathcal{Q}_7)$$

over the YAGO2s dataset. We instantiate p as follows.

P_1 : single inverse wavefront $USA \rightarrow ?p$.

P_2 : single direct wavefront $?p \rightarrow USA$.

P_3 : two wavefronts

$$?p \rightarrow :locatedIn+/:dealsWith+ \leftarrow USA.$$

P_4 : P_2 but with a threaded sub-path

$$:locatedIn+/:dealsWith+ USA.$$

Fig. 15a shows the effect of wavefront choice on search cardinality. Note the order of magnitude difference between the best, P_4 , versus the worst, P_1 . The three types of redundancy pruning—**cache**, **delta**, and **fpp**—are illustrated for each plan. Fig. 15b plots search size across iterations for P_2 with pruning; over 40% of tuples are pruned! Fig. 15c plots delta sizes over iterations for P_1 and P_3 . Note how the selective search of P_3 is better behaved than the rapid expansion of P_1 . In Fig. 15d, the total execution time for each plan is presented. This demonstrates the significant improvement in performance achievable by careful design of the plan.

5.3 Comparison with Other Systems

We design a microbenchmark to analyze the performance of the WAVEGUIDE prototype in comparison with two RDF stores: VIRTUOSO [12] and JENA TDB [8]. Both VIRTUOSO and JENA TDB are well known and widely used today to store and query RDF data. VIRTUOSO is an example of an RDF store which is backed by a relational database, which makes it similar to WAVEGUIDE prototype. On the other hand, JENA TDB is a native solution developed specifically to handle graph data.

We design three experiments which aim to benchmark two main aspects of property path evaluation: 1) transitive closure computation and 2) query planning.

Transitive Closure. Computation of transitive closure is one of the main operations which are encountered during evaluation of property paths. Hence, it is important to handle this operation efficiently in order to be able to answer property path queries quickly. Our benchmark consists of a number of realistic simple transitive closure queries over YAGO2s dataset which follow the template:

$$?entity :locatedIn+ place \quad (\mathcal{Q}_8)$$

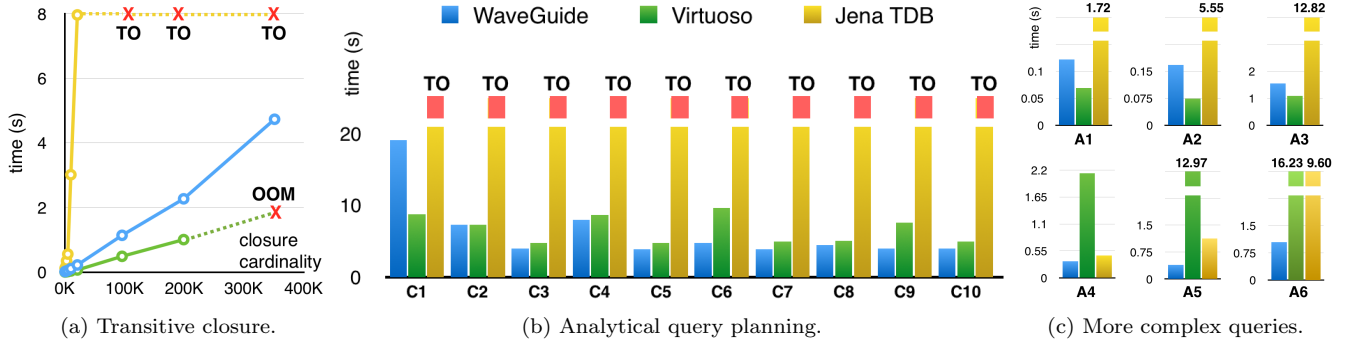


Figure 16: Benchmarking vs. state of the art.

Here, $\mathcal{Q}_8$ finds all entities transitively located in a certain place, which is given as a query constant. By varying this constant, we design transitive closure queries with increasing answer cardinalities, which allow us to benchmark the efficiency of transitive closure computation in a given system.

The benchmark results are presented on Fig.16a. VIRTUOSO has the fastest transitive closure computation engine. This is expected as VIRTUOSO is based on high-performance relational database written in C. Moreover, all its transitive closure computations are performed completely in main memory without spilling to disk. However, this can be a serious performance bottleneck for large cardinality property path queries, where VIRTUOSO runs out of memory (OOM). WAVEGUIDE prototype is roughly two times slower than VIRTUOSO. While our prototype is also based on a high-performance relational database, POSTGRESQL, we utilize stored procedures, which are less efficient than inline SQL or the in-memory computations used by VIRTUOSO. Lastly, JENA TDB did not scale well for transitive closures, timing out (TO) on all but the smallest cardinality queries.

Query Planning. This experiment aims to highlight the importance of planning in evaluation of property path queries. We design a number of queries over YAGO2s dataset which are based on the following template:

$$\text{select distinct ?x ?y \{ } \quad (\mathcal{Q}_9)$$

$$\quad ?x : \text{locatedIn+ } place . \quad [\mathcal{Q}_8]$$

$$\quad ?x (a/b/c)+ ?y . \} \quad [\mathcal{Q}_6]$$

Observe that $\mathcal{Q}_9$ is a conjunction of templates $\mathcal{Q}_8$ and $\mathcal{Q}_6$. We could not use just $\mathcal{Q}_6$ because of VIRTUOSO's limitation requiring property path queries to be bound on at least one variable. In $\mathcal{Q}_9$, that binding is done by $\mathcal{Q}_8$.

First, we set *place* in $\mathcal{Q}_8$ to :Earth, to simulate an *analytical* workload by placing a *loose* binding on ?x in property path $\mathcal{Q}_6$. Second, we set values of *a*, *b* and *c* to those used in queries Y1–Y10 in §5.2 to produce queries C1–C10. The benchmark results are presented in Fig.16b. We observe that JENA TDB did not finish in the allotted time frame (timed out, TO) for any of the queries. We attribute this to JENA TDB's limited scalability of transitive closure computations. On the other hand, despite having a faster transitive closure computation engine, VIRTUOSO is slower than WAVEGUIDE in many of the queries, sometimes significantly. We attribute this to the limited plan space considered by VIRTUOSO, which requires transitive queries to be bound by one of the variables. VIRTUOSO was evaluating $\mathcal{Q}_8$ first, and

then *pipelining* the results to $\mathcal{Q}_6$. As discussed in §5.2, such pipelined plans often do not perform well for loosely bound queries.

Query Planning vs. Transitive Closure. The goal of our third experiment is to understand the capacity in which the efficiency of transitive closure and query planning affect query evaluation. We design a set of six real-world queries over YAGO2s dataset which we split into two groups. Queries in the first group (A1–A3) are simple as they contain one or two non-transitive and one transitive predicate. Therefore, efficient transitive closure computation plays a bigger role than query planning for this group. Queries in the second group are more complex as they either contain two or more transitive (A5, A6) or a single transitive and three non-transitive predicates (A4). For these, a good evaluation plan is more important.

The benchmark results are presented in Fig. 16c. Due to the simplicity of the queries in the first group, all three engines used identical plans to evaluate them. VIRTUOSO, with the fastest transitive closure, had the lowest evaluation time. For queries in the second group, WAVEGUIDE was the fastest engine for all of the queries. The slight increase in query complexity from single (A4) to two or more transitive predicates (A5, A6) allowed WAVEGUIDE to gain a significant advantage over the two other systems. This shows the importance of the richer plan space considered by WAVEGUIDE.

6. CONCLUSIONS

We have devised a rich domain of evaluation plans for property-path queries (RPQs) in SPARQL, and have shown it extends significantly over the state of the art. WAVEGUIDE's plans, which encompasses powerful optimization techniques, offer a solid step towards building a full-fledged, cost-based query optimizer for SPARQL with property paths. Our next steps in this project are to construct an *enumerator* that can enumerate efficiently through the WP space for a given query and to devise a *cost estimator* for WPs so the better plans can be identified. For the cost estimator, we need to determine an array of graph *statistics* that can be evaluated offline that enable estimation that is good enough to distinguish between good and bad plans.

The rising prominence of graph data raises many challenges. As the community rose in the past to accomplish the “impossible” for relational data, for semi-structured, and for unstructured search, we too will meet these new challenges.

7. REFERENCES

- [1] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of databases*, volume 8. Addison-Wesley, 1995.
- [2] S. Abiteboul, D. Quass, J. McHugh, J. Widom, and J. L. Wiener. The Lorel query language for semistructured data. *International Journal on Digital Libraries*, 1(1):68–88, 1997.
- [3] R. Agrawal. Alpha: An extension of relational algebra to express a class of recursive queries. *Software Engineering, IEEE Transactions on*, 14(7):879–885, 1988.
- [4] P. Barcelo, L. Libkin, A. W. Lin, and P. T. Wood. Expressive languages for path queries over graph-structured data. *Transactions on Database Systems*, 37(4):31, 2012.
- [5] M. Bojanczyk. Automata for data words and data trees. In *RTA*, pages 1–4, 2010.
- [6] P. Buneman, S. Davidson, G. Hillebrand, and D. Suciu. A query language and optimization techniques for unstructured data. *SIGMOD Record*, 25(2):505–516, 1996.
- [7] D. Calvanese, G. De Giacomo, M. Lenzerini, and M. Y. Vardi. Rewriting of regular expressions and regular path queries. In *Proceedings of the Symposium on Principles of Database Systems*, pages 194–204. ACM, 1999.
- [8] J. J. Carroll, I. Dickinson, C. Dollin, D. Reynolds, A. Seaborne, and K. Wilkinson. Jena: implementing the semantic web recommendations. In *Proceedings of the 13th international World Wide Web conference on Alternate track papers & posters*, pages 74–83. ACM, 2004.
- [9] M. P. Consens, A. O. Mendelzon, D. Vista, and P. T. Wood. Constant propagation versus join reordering in datalog. In *Rules in Database Systems*, pages 245–259. Springer, 1995.
- [10] The DBpedia knowledge base. <http://dbpedia.org/>.
- [11] S. Dey, V. Cuevas-Vicentín, S. Köhler, E. Gribkoff, M. Wang, and B. Ludäscher. On implementing provenance-aware regular path queries with relational query engines. In *Proceedings of the Joint EDBT/ICDT 2013 Workshops*, pages 214–223. ACM, 2013.
- [12] O. Erling and I. Mikhailov. Virtuoso: RDF Support in Native RDBMS. *Semantic Web Information Management*, 1:501, 2010.
- [13] D. Florescu, A. Y. Levy, and A. O. Mendelzon. Database techniques for the world-wide web: A survey. *SIGMOD Record*, 27(3):59–74, 1998.
- [14] S. Harris and A. Seaborne. SPARQL 1.1 query language. W3C working draft. <http://www.w3.org/TR/sparql11-query/>, November 2012.
- [15] K. J. Kochut and M. Janik. Sparqler: Extended sparql for semantic association discovery. In *The Semantic Web: Research and Applications*, pages 145–159. Springer, 2007.
- [16] A. Koschmieder and U. Leser. Regular path queries on large graphs. In *Scientific and Statistical Database Management*, pages 177–194. Springer Berlin Heidelberg, 2012.
- [17] K. Losemann and W. Martens. The complexity of evaluating path expressions in SPARQL. In *Proceedings of the 31st symposium on Principles of Database Systems*, pages 101–112. ACM, 2012.
- [18] A. Mendelzon and P. Wood. Finding regular simple paths in graph databases. *SIAM Journal on Computing*, 24(6):1235–1258, 1995.
- [19] J. Pérez, M. Arenas, and C. Gutierrez. nSPARQL: A navigational language for rdf. *Web Semantics: Science, Services and Agents on the World Wide Web*, 8(4):255–270, 2010.
- [20] E. Prud’Hommeaux, A. Seaborne, et al. SPARQL query language for RDF. *W3C Recommendation*, 15, 2008.
- [21] W3C: Resource Description Framework (RDF). <http://www.w3.org/TR/rdf-concepts/>, 2004.
- [22] L. Segoufin. Automata and logics for words and trees over an infinite alphabet. In *Computer Science Logic*, pages 41–57. Springer, 2006.
- [23] L. Segoufin. Static analysis of XML processing with data values. *ACM Sigmod Record*, 36(1):31–38, 2007.
- [24] YAGO2s: A high-quality knowledge base. <http://yago-knowledge.org/resource/>. Max Planck Institut Informatik.
- [25] N. Yakovets, P. Godfrey, and J. Gryz. Evaluation of SPARQL property paths via recursive SQL. In L. Bravo and M. Lenzerini, editors, *AMW*, volume 1087 of *CEUR Workshop Proceedings*. CEUR-WS.org, May 2013.
- [26] N. Yakovets, P. Godfrey, and J. Gryz. WAVEGUIDE: evaluating SPARQL property path queries. In *Proceedings of the 18th International Conference on Extending Database Technology, EDBT 2015, Brussels, Belgium, March 23-27, 2015.*, pages 525–528, 2015.
- [27] H. Zauner, B. Linse, T. Furche, and F. Bry. A RPL through RDF: expressive navigation in RDF graphs. In *Web Reasoning and Rule Systems*, pages 251–257. Springer, 2010.

APPENDIX

A. SYSTEM SETUP

A.1 Architecture

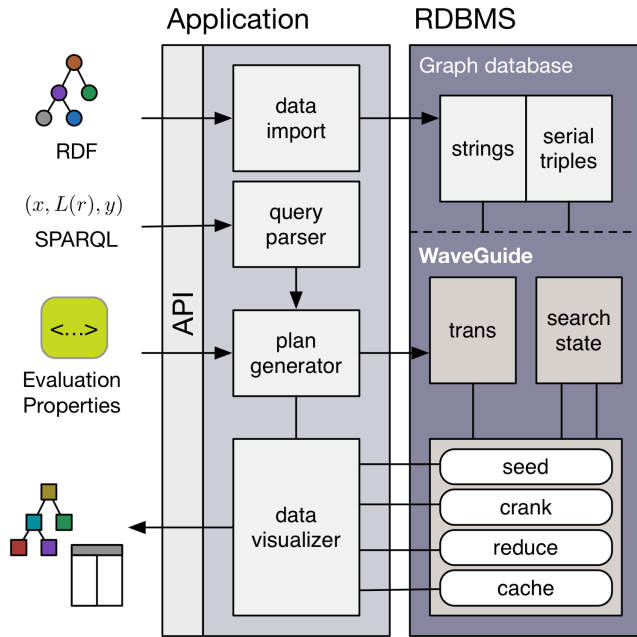


Figure 17: Overview of the prototype system.

A.2 Hardware

Our benchmark was executed on a host 2xXeon E5-2640v2 CPU server with 10x7200RPM HDDs running CentOS 6. PostgreSQL, OpenLink Virtuoso and Jena TDB were executed in their own virtual machines which were given 4 CPU cores, 4GB of RAM and 1TB of HDD space.

A.3 Software

WAVEGUIDE prototype planner was implemented in Java 1.8. WAVEGUIDE database execution engine was based on PostgreSQL 9.3. VIRTUOSO was built against open source distribution of version 07.20.3215-pthreads (latest at the time of submission of this paper) which is maintained by OpenLink on GitHub. Similarly, we obtained the latest version (3.0.1) of JENA TDB for our tests. Both DBpedia and YAGO2s were fully indexed on all systems to ensure indexed access paths for all queries used in benchmarks. In the transitive closure experiments, we limited the amount of RAM available to each system to 100MB. In VIRTUOSO, we have modified the source code to change the default transitive memory limit from 1GB to 100MB. This limit was raised to 4GB in the rest of the experiments, since all available memory in the VM (4GB) was allowed to be used for all systems. OOM abbreviation means the system ran out of allocated memory while executing a given query. For all queries in §5.3, a timeout (TO) was set to 10 minutes.

B. QUERIES

B.1 Threading

L	p	p ₁	p ₂
1	byTransport	rdf:type	owl:disjointWith
2	happenedIn	hasCapital	created
3	happenedIn	hasCapital	owns
4	happenedIn	hasCapital	rdf:type
5	happenedIn	hasCapital	participatedIn
6	happenedIn	dealsWith	rdf:type
7	happenedIn	dealsWith	hasCapital
8	happenedIn	rdf:type	owl:disjointWith
9	hasWonPrize	rdf:type	owl:disjointWith
10	hasWordnetDomain	rdf:type	owl:disjointWith
11	isCitizenOf	hasCapital	created
12	isCitizenOf	hasCapital	owns
13	isCitizenOf	hasCapital	rdf:type
14	isCitizenOf	hasCapital	participatedIn
15	isCitizenOf	dealsWith	rdf:type
16	isCitizenOf	dealsWith	hasCapital
17	isCitizenOf	rdf:type	owl:disjointWith
18	isLocatedIn	hasCapital	created
19	isLocatedIn	hasCapital	owns
20	isLocatedIn	hasCapital	rdf:type
21	isLocatedIn	dealsWith	hasCapital
22	isLocatedIn	rdf:type	owl:disjointWith
23	isPoliticianOf	hasCapital	created
24	isPoliticianOf	hasCapital	owns
25	isPoliticianOf	hasCapital	rdf:type
26	isPoliticianOf	hasCapital	participatedIn
27	isPoliticianOf	rdf:type	owl:disjointWith
28	rdfs:subClassOf	rdf:type	owl:disjointWith
29	wasBornIn	hasCapital	created
30	wasBornIn	hasCapital	owns
31	wasBornIn	hasCapital	rdf:type
32	wasBornIn	hasCapital	participatedIn
33	wasBornIn	dealsWith	rdf:type
34	wasBornIn	dealsWith	hasCapital
35	wasBornIn	rdf:type	owl:disjointWith
36	created	hasChild	isMarriedTo
37	created	influences	isMarriedTo
38	created	influences	hasChild
39	created	isMarriedTo	hasChild
40	hasChild	influences	isMarriedTo
41	hasChild	influences	hasChild
42	hasChild	isMarriedTo	hasChild
43	hasChild	hasChild	isMarriedTo
44	influences	hasChild	isMarriedTo
45	influences	isMarriedTo	hasChild
46	influences	influences	isMarriedTo
47	influences	influences	hasChild
48	isKnownFor	hasChild	isMarriedTo
49	isKnownFor	isMarriedTo	hasChild
50	isKnownFor	influences	isMarriedTo
51	isKnownFor	influences	hasChild
52	isMarriedTo	influences	isMarriedTo
53	isMarriedTo	influences	hasChild
54	isMarriedTo	hasChild	isMarriedTo
55	isMarriedTo	isMarriedTo	hasChild

B.2 Loop Caching

D	a	b
1	formerTeam	name
2	region	candidate
3	clubs	currentMember
4	starring	title
5	successor	predecessor
6	successor	before
7	associatedMusicalArtist	currentMembers

Loop Caching (continued)

D	<i>a</i>	<i>b</i>
8	associatedBand	currentMembers
9	associatedActs	currentMembers
10	after	predecessor
11	successor	successor
12	before	successor
13	before	successor
14	successor	after
15	currentMembers	associatedBand
16	before	predecessor
17	after	successor
18	imageCaption	ordo
19	imageCaption	order
20	label	subdivisionName
21	candidate	region

B.3 Partial Loop Caching

Y/C	<i>a</i>	<i>b</i>	<i>c</i>
1	isLocatedIn	hasCapital	owns
2	isConnectedTo	isLocatedIn	owns
3	dealsWith	participatedIn	isLocatedIn
4	isLocatedIn	owns	created
5	participatedIn	isLocatedIn	dealsWith
6	hasCapital	isLocatedIn	dealsWith
7	happenedIn	hasCapital	participatedIn
8	owns	isConnectedTo	isLocatedIn
9	dealsWith	hasCapital	isLocatedIn
10	participatedIn	happenedIn	hasCapital
Y	<i>a</i>	<i>b</i>	<i>c</i>
11	owns	isLocatedIn	dealsWith
12	hasCapital	participatedIn	happenedIn
13	isLeaderOf	dealsWith	participatedIn
14	dealsWith	owns	isLocatedIn
15	created	isLocatedIn	owns
16	hasCapital	owns	isLocatedIn
17	owns	isLocatedIn	hasCapital
18	hasAcademicAdvisor	isInterestedIn	influences
19	isInterestedIn	influences	hasAcademicAdvisor
20	isMarriedTo	hasChild	influences
21	influences	hasAcademicAdvisor	isInterestedIn
22	isMarriedTo	influences	hasChild
23	hasChild	influences	isMarriedTo
24	influences	isMarriedTo	hasChild
25	owns	created	isLocatedIn

B.4 State of the art Planning

A	query
1	select distinct ?p {?p :diedIn/:isLocatedIn+ :France .}
2	select distinct ?p {?p :wasBornIn/:isLocatedIn+ :France .}
3	select distinct ?p {?p :wasBornIn/:isLocatedIn+/:type :Europe .}
4	select distinct ?p {?p :isMarriedTo/:diedIn/:isLocatedIn+/:dealsWith :United_States .}
5	select distinct ?p {?p :isMarriedTo/:diedIn/:isLocatedIn+/:dealsWith/:isLocatedIn+ :Earth .}
6	select distinct ?p {?p :livesIn/(owl:sameAs*/:locatedIn)+/owl:sameAs*/:dealsWith/ (owl:sameAs*/:locatedIn)+/owl:sameAs* :Earth .}